Kernel Data Structures

F.CSM302 Үйлдлийн систем · Бие даалт 1 · Хуваарилалтын код: 4

Гишүүд: [нэр 1], [нэр 2], [нэр 3] (бөглөнө)

Хураангуй

Асуудал: kernel-д ашиглагддаг холбоост жагсаалт, дараалал, стек, bitmap, мод зэрэг өгөгдлийн бүтцийн ажиллагааг ойлгох. **Шийдэл:** `list_head`-ийн next/prev семантикийг C-ээр дахин хэрэгжүүлж, нэг командын хэл бүхий CLI ба түүнтэй ижил хөдөлгүүртэй interactive UI (Kernel structure graph) бүтээв; бодит kernel module-ийг мөн бичсэн. **Туршилт:** 3 жишээ + 5000 давталттай stress тест ASan/UBSan-тай давав; UI ба C-ийн гаралт 3 скриптэд бүрэн давхцсан. **Гол үр дүн:** оролт, огноо, хязгаарыг шалгадаг, хэрэглэгч өөр өгөгдлөөр дахин туршиж болох хэрэгсэл.

1. Удиртгал

Үйлдлийн системийн kernel нь процессын жагсаалт, ready queue, bitmap (аль нэг нөөц чөлөөтэй эсэх), модон бүтэц (scheduler, файл систем) зэргийг өгөгдлийн бүтцээр удирддаг. Төслийн зорилго нь эдгээрийн үндсэн үйлдлийг хэрэгжүүлж, нүдэнд харагдахуйц болгох. Хамрах хүрээ: хэрэглэгчийн орчны загвар + нэг kernel module; бодит scheduler-д холбохгүй.

2. Сурах бичгийн шаардлага

Programming Project (Kernel Data Structures): (1) `struct birthday { day, month, year; struct list_head list; }` тодорхойлох; (2) 5 жишээ үүсгэж `list_add_tail()`-ээр жагсаалтад нэмэх; (3) `list_for_each_entry`-ээр уншиж kernel log-д хэвлэх; (4) module унтраахад `list_del` ба `kfree`-ээр бүх элементийг чөлөөлөх. Хоёрдугаар хэсэгт Linux kernel-ийн queue, stack, bitmap, tree-ийг судлан хэрэгжүүлэх. (Тайлбар: шаардлагыг ерөнхий мэдлэгээс бичсэн тул багшийн хуваарилсан яг найруулгатай тулгаж баталгаажуулна уу.)

3. Онолын үндэс

list_head – intrusive, давхар холбоост, дугуй жагсаалт. Node өөрөө өгөгдлийн struct дотор `list_head` талбар агуулж, `container_of` макроор эзэн struct-ийг сэргээнэ. Нэмэх/устгах нь O(1), хайх нь O(n). **Queue** (FIFO) – tail-д нэмж, head-ээс авна. **Stack** (LIFO) – head-д нэмж, head-ээс авна. **Bitmap** – 1 bit = 1 нөөц; set/clear/test O(1). **Binary search tree** – зүүн < үндэс < баруун; дундаж O(log n), хамгийн муу O(n) (kernel-д тэнцвэржүүлсэн rbtree ашигладаг). Нийтлэг алдаа: list_del-ийн дараа pointer-ыг ашиглах, node-ыг чөлөөлөхгүй үлдээх (leak), iterate хийж байхдаа устгах (`_safe` хувилбар хэрэгтэй).

4. Системийн зохиомж

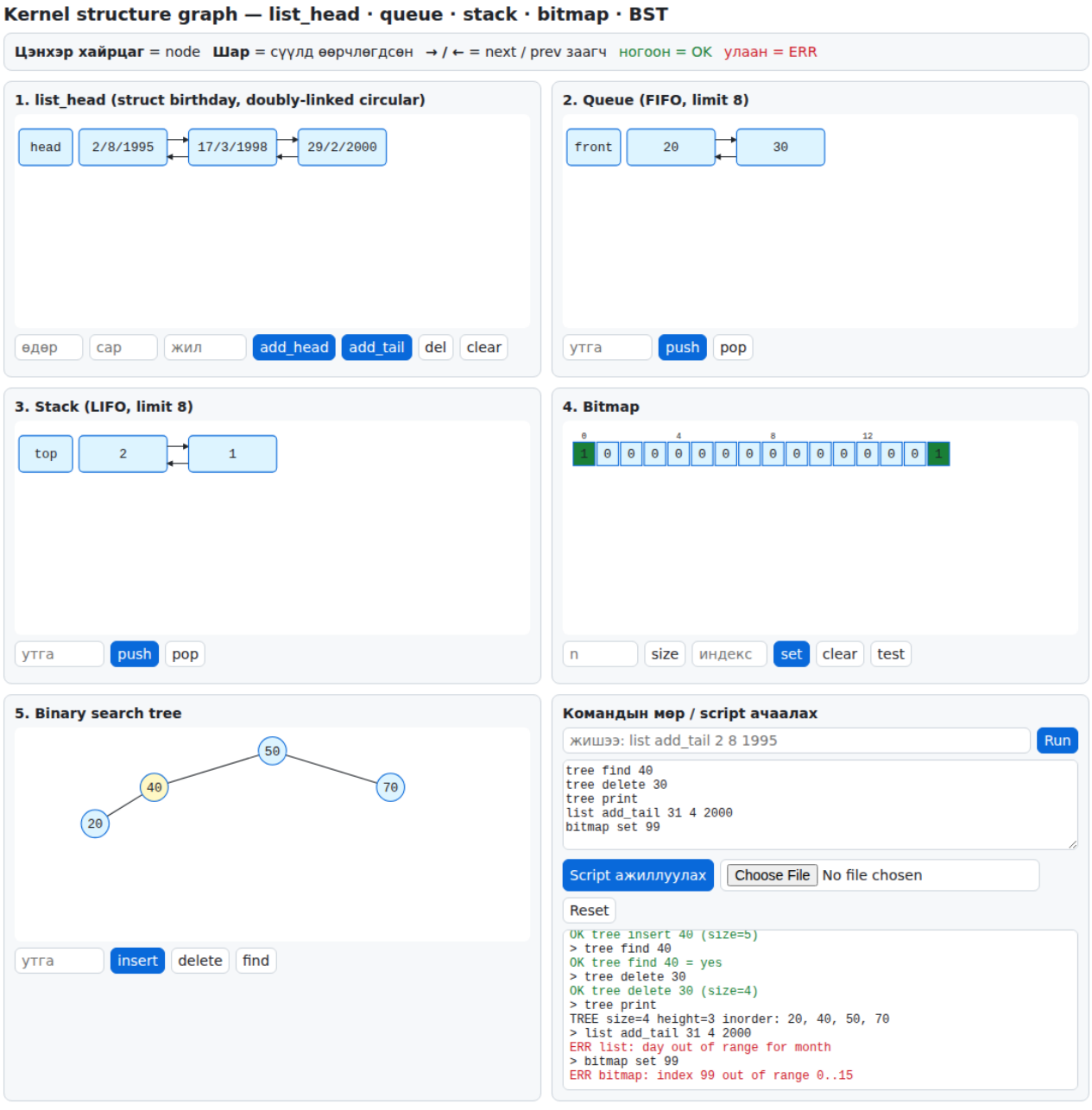
Компонент: (a) **kds.c** – командын интерпретатор + 5 бүтэц; (b) **index.html** – ижил команд, мөн ижил хөдөлгүүр, SVG график; (c) **kds_module.c** – kernel орчны хувилбар. Өгөгдлийн урсгал: хэрэглэгчийн команд → интерпретатор → бүтцийн төлөв шинэчлэх → мессеж (OK/ERR) → (UI дээр) төлөвөөс график зурах. График нь тусдаа өгөгдөл биш, шууд бүтцийн төлөвөөс зурагддаг тул бодит ажиллагаатай үргэлж нийцнэ.

5. Хэрэгжилт

Хэл: C (gcc, -fsanitize=address,undefined), JavaScript (SVG), kernel C. Гол шийдлүүд: (1) list.h-ийн `__list_add`, `list_add`, `list_add_tail`, `list_del`, `container_of`-ийг өөрөө бичсэн; (2) queue ба stack нь ижил `list_head`-ийг хуваалцаж зөвхөн нэмэх талаараа ялгарна; (3) огноо шалгалт нь хаврын жилийг (4-т хуваагдах, 100-д биш, эсвэл 400) тооцно; (4) модны устгал нь хоёр хүүтэй үед баруун дэд модны хамгийн бага утгаар солино; (5) kernel module-д `list_for_each_entry_safe`, kmalloc алдаа гарвал цэвэрлэх замтай.

6. Туршилт ба UI дүрслэл

UI-д 5 карт бий: list (next/prev сум), queue, stack, bitmap (bit тус бүр нүд), tree (зангилаа ба холбоос). Хэрэглэгч огноо, тоо, bitmap-ийн хэмжээг өөрчилж, товч эсвэл командаар үйлдэл хийнэ; шар өнгө сүүлийн өөрчлөлтийг, консол OK/ERR-ийг харуулна. Reset ба script ачаалах боломжтой.



7. Туршилтын арга

Орчин: Ubuntu 24 (sandbox), gcc, node, Chromium (playwright). Тест: (a) examples/textbook.txt – сурах бичгийн дүр; (b) examples/all_structures.txt – бусад бүтэц; (c) examples/errors.txt – захын ба буруу оролт; (d) 5000 push/pop stress – ERR гарахгүй, ASan leak/UB мэдээлэхгүй; (e) UI-ийн хөдөлгүүрийг node-оор ажиллуулж C-ийн гаралттай `diff`-лэх. Хүлээгдэх үр дүн нь tests/expected_*.out-д хадгалагдсан.

8. Үр дүн

Тест	Үр дүн
textbook	PASS
all_structures	PASS
errors	PASS
stress (5000×4 үйлдэл)	PASS, ERR 0
UI == C (3 script)	PASS ×3
Kernel module	Compile хийгдээгүй (Linux VM шаардлагатай)

Сурах бичгийн жишээний бодит гаралт:

```
OK list add_tail 2/8/1995 (size=1)
OK list add_tail 17/3/1998 (size=2)
OK list add_tail 29/2/2000 (size=3)
OK list add_tail 5/11/2001 (size=4)
OK list add_tail 30/12/2003 (size=5)
LIST size=5: 2/8/1995 -> 17/3/1998 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 17/3/1998 (size=4)
LIST size=4: 2/8/1995 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 2/8/1995 (size=3)
OK list del 29/2/2000 (size=2)
OK list del 5/11/2001 (size=1)
OK list del 30/12/2003 (size=0)
LIST size=0: (empty)
```

Буруу оролтын бодит гаралт (examples/errors.txt):

```
ERR list: day out of range for month
ERR list: day out of range for month
ERR list: month must be 1..12
ERR list: year must be 1900..2100
ERR list: usage <op> <day> <month> <year>
ERR list: 1/1/2000 not found
LIST size=0: (empty)
ERR list: missing operation
ERR unknown structure 'frob'
ERR queue: underflow (empty)
ERR stack: underflow (empty)
ERR queue: usage push <int>
ERR bitmap: size must be 1..128
ERR bitmap: size must be 1..128
ERR bitmap: index 99 out of range 0..31
ERR bitmap: index -1 out of range 0..31
OK tree insert 5 (size=1)
ERR tree: duplicate 5
ERR tree: 9 not found
```

9. Хэлэлцүүлэг

Бүх буруу оролт тодорхой мессеж өгч програм унасангүй. Хязгаарлалт: (1) kernel module-ийг sandbox-д ачаалж чадаагүй тул dmesg-ийн гаралтыг энэ тайланд screenshot-оор нотлоогүй – багийн гишүүд VM дээр гүйцэтгэж энд нэмэх ёстой; (2) UI болон C нь тусдаа хэрэгжилт (дахин бичсэн), ижил байдлыг тестээр баталсан боловч нэг эх кодоос үүсээгүй; (3) модыг тэнцвэржүүлээгүй; (4) queue/stack-ийн багтаамж

тогтмол 8; (5) GitHub дахь commit түүхийг гишүүд өөрсдөө үүсгэнэ.

10. Дүгнэлт

list_head, queue, stack, bitmap, tree-г хэрэгжүүлж, ижил командын хэлээр CLI ба график UI-аас удирдах боломжтой болгосон. Зөв ажиллагааг 3 скрипт, stress тест, UI==C харьцуулалтаар нотолсон. Үлдсэн ажил: kernel module-ийг VM-д туршиж screenshot нэмэх, GitHub release (9-р долоо хоног).

Ашигласан материал

Silberschatz, Galvin, Gagne – Operating System Concepts (Programming Projects); Linux kernel: include/linux/list.h, kfifo.h, bitmap.h баримт; AI хэрэгсэл: Claude (Anthropic) – кодын эх ба тайлангийн ноорог бэлтгэхэд ашигласан.